

Output:

1. Sample Case study on class & objects (CO1):

Bank Account Management System:

Problem Statement: Design a system to manage bank accounts.

Classes: Account, Savings-Account, Current-Account.

Attributes: Account number, balance, account type, interest rate (for savings account), overdraft limit (for current account), etc.

Methods: Deposit, Withdraw, Calculate Interest, Transfer, etc.

Object Usage: Create objects for each customer's account and perform transactions like deposits, withdrawals, and transfers.

Steps for Execution:

1. Create a folder named BankManagement.
2. Inside it, create a folder named record.
3. Save the Java program as BankManagement.java inside the record folder.
4. Open the BankManagement folder in VS code/Eclipse
5. If there are no compilation errors, execute the program.
6. The bank menu will be displayed.
7. Enter the required menu choice.
8. The selected banking operation will be performed.
9. The menu will be displayed again until option 6 is selected.
10. Select option 6 to terminate the program.
11. The program execution is completed.

Algorithm :

Step 1: Start the program.

Step 2: Import the Scanner class.

Step 3: Create the parent class Account with account number, balance, and account type.

Step 4: Define methods in Account for:

- Deposit
- Withdraw
- Transfer money

Output:

- Display account details

Step 5: Create SavingsAccount class by extending Account.

Step 6: Add the interestRate attribute and define the calculateInterest() method.

Step 7: Create CurrentAccount class by extending Account.

Step 8: Add the overdraftLimit attribute and override the withdraw() method.

Step 9: In the main() method, create objects for SavingsAccount and CurrentAccount.

Step 10: Display the bank menu.

Step 11: Read the user's choice.

Step 12: Perform the selected operation:

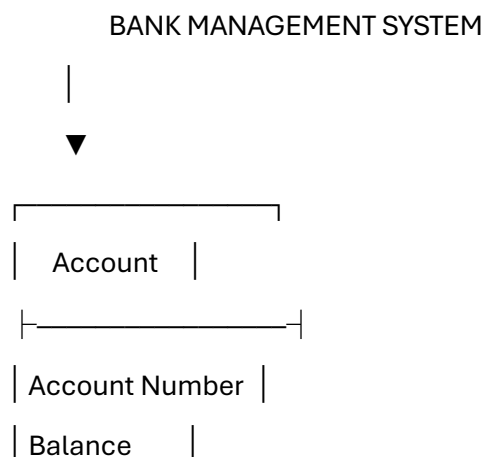
- If choice is 1, deposit money.
- If choice is 2, withdraw money.
- If choice is 3, calculate interest.
- If choice is 4, transfer money.
- If choice is 5, display account details.
- If choice is 6, exit the program.
- Otherwise, display an invalid-choice message.

Step 13: Repeat the menu using a do-while loop until the user selects option 6.

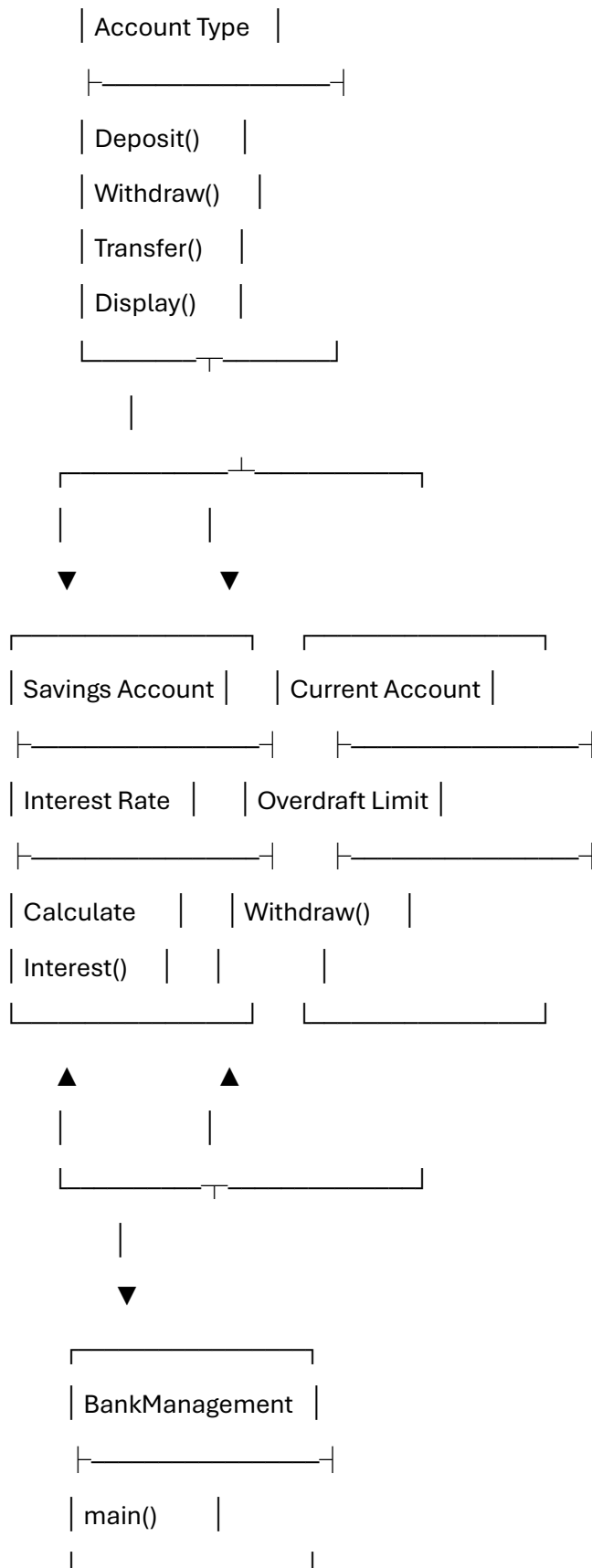
Step 14: Close the Scanner.

Step 15: Stop the program.

UML Class Diagram:



Output:



Output:

Program:

```
package record;

import java.util.Scanner;

//Parent Class

class Account {

    int accountNumber;

    double balance;

    String accountType;

    Account(int accountNumber, double balance, String accountType) {

        this.accountNumber = accountNumber;

        this.balance = balance;

        this.accountType = accountType;

    }

    void deposit(double amount) {

        balance = balance + amount;

        System.out.println("Amount deposited: " + amount);

    }

    void withdraw(double amount) {

        if (amount <= balance) {

            balance = balance - amount;

            System.out.println("Amount withdrawn: " + amount);

        }

        else {

            System.out.println("Insufficient balance");

        }

    }

}
```

Output:

```
void transfer(Account receiver, double amount) {
    if (amount <= balance) {
        balance = balance - amount;
        receiver.balance = receiver.balance + amount;
        System.out.println("Transfer successful: " + amount);
    } else {
        System.out.println("Insufficient balance");
    }
}

void display() {
    System.out.println("Account Number: " + accountNumber);
    System.out.println("Account Type: " + accountType);
    System.out.println("Balance: " + balance);
}

//Savings Account
class SavingsAccount extends Account {
    double interestRate;

    SavingsAccount(int accountNumber, double balance, double interestRate) {
        super(accountNumber, balance, "Savings");
        this.interestRate = interestRate;
    }

    void calculateInterest() {
        double interest = balance * interestRate / 100;
        balance = balance + interest;
        System.out.println("Interest added: " + interest);
    }
}
```

Output:

```
}
```

```
}
```

```
//Current Account
```

```
class CurrentAccount extends Account {
```

```
    double overdraftLimit;
```

```
    CurrentAccount(int accountNumber, double balance, double overdraftLimit) {
```

```
        super(accountNumber, balance, "Current");
```

```
        this.overdraftLimit = overdraftLimit;
```

```
    }
```

```
    void withdraw(double amount) {
```

```
        if (amount <= balance + overdraftLimit) {
```

```
            balance = balance - amount;
```

```
            System.out.println("Amount withdrawn: " + amount);
```

```
        } else {
```

```
            System.out.println("Overdraft limit exceeded");
```

```
        }
```

```
    }
```

```
}
```

```
//Main Class
```

```
public class BankManagement {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        SavingsAccount s1 =
```

```
            new SavingsAccount(101, 5000, 5);
```

Output:

```
CurrentAccount c1 =
```

```
    new CurrentAccount(102, 3000, 1000);
```

```
int choice;
```

```
do {
```

```
    System.out.println("\n===== BANK MENU =====");
```

```
    System.out.println("1. Deposit");
```

```
    System.out.println("2. Withdraw");
```

```
    System.out.println("3. Calculate Interest");
```

```
    System.out.println("4. Transfer Money");
```

```
    System.out.println("5. Display Account Details");
```

```
    System.out.println("6. Exit");
```

```
    System.out.print("Enter your choice: ");
```

```
    choice = sc.nextInt();
```

```
switch (choice) {
```

```
    case 1:
```

```
        s1.deposit(1000);
```

```
        System.out.println("Amount deposited.");
```

```
        break;
```

```
    case 2:
```

```
        s1.withdraw(500);
```

```
        System.out.println("Amount withdrawn.");
```

```
        break;
```

```
    case 3:
```

```
        s1.calculateInterest();
```

Output:

```
System.out.println("Interest calculated.");
```

```
break;
```

case 4:

```
s1.transfer(c1, 500);
```

```
System.out.println("Amount transferred.");
```

```
break;
```

case 5:

```
System.out.println("\nSavings Account:");
```

```
s1.display();
```

```
System.out.println("\nCurrent Account:");
```

```
c1.display();
```

```
break;
```

case 6:

```
System.out.println("Thank you!");
```

```
break;
```

default:

```
System.out.println("Invalid choice");
```

```
}
```

```
} while (choice != 6);
```

```
sc.close();
```

```
}
```

```
}
```


Output:

```
===== BANK MENU =====
1. Deposit
2. Withdraw
3. Calculate Interest
1. Transfer Money
3. Display Account Details
3. Exit
Enter your choice: 1
Amount deposited: 1000.0
Amount deposited.

===== BANK MENU =====
1. Deposit
2. Withdraw
3. Calculate Interest
1. Transfer Money
3. Display Account Details
3. Exit
Enter your choice: 2
Amount withdrawn: 500.0
Amount withdrawn.

===== BANK MENU =====
1. Deposit
2. Withdraw
3. Calculate Interest
1. Transfer Money
3. Display Account Details
3. Exit
Enter your choice: 3
Interest added: 275.0
Interest calculated.

===== BANK MENU =====
1. Deposit
2. Withdraw
3. Calculate Interest
1. Transfer Money
3. Display Account Details
3. Exit
Enter your choice: 4
Transfer successful: 500.0
Amount transferred.
```